Modul 6 Inheritance

Tujuan Praktikum
1. Praktikan dapat mengerti dan memahami inheritance secara mendalam. 2. Praktikan dapat memahami konsep inheritance dan polymorfisme serta dapat mengimplementasikan dalam suatu program..

6.1 Pendahuluan

Pada dasarnya Kita sebagai manusia sudah terbiasa untuk melihat objek yang berada di sekitar Kita tersusun secara hierarki berdasarkan kelas-nya masing-masing. Dari sini kemudian timbul suatu konsep tentang pewarisan yang merupakan suatu proses dimana suatu *class* diturunkan dari kelas lainnya sehingga ia mendapatkkan ciri atau sifat dari kelas tersebut.

Dari hierarki di atas, semakin ke bawah, kelas akan semakin bersifat spesifik. Misalnya, sebuah Kelas Pria memiliki seluruh sifat yang dimiliki oleh manusia, demikian halnya juga Wanita.

Penurunan sifat tersebut dalam Bahasa Pemrograman Java disebut dengan *Inheritance* yaitu satu dalam Pilar Dasar OOP (*Object Oriented Programing*), yang dalam implementasinya merupakan sebuah hubungan "adalah bagian dari" atau *"is a relationship" Object* yang di-*inherit* (diturunkan).

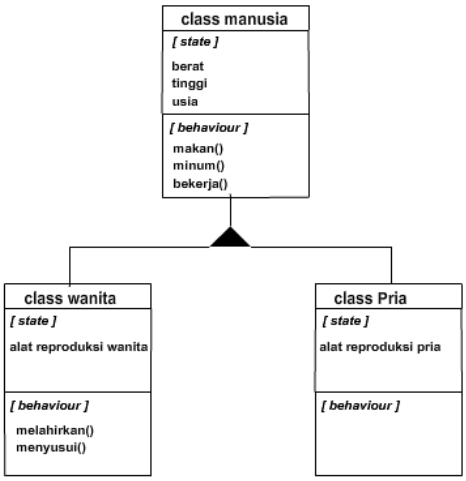
Latar belakang diperlukannya suatu *inheritance* dalam pemrograman Java adalah untuk **menghindari duplikasi *Object*** baik itu *field*, variabel maupun *method* yang sebenarnya merupakan *Object* yang bisa diturunkan dari hanya sebuah kelas. Jadi *inheritance* bukan sebuah Kelas yang diturunkan oleh sebuah *Literial*, tapi lebih menunjukkan ke hubungan antar *Object* itu sendiri.

Sedangkan suatu kemampuan dari sebuah *Object* untuk membolehkan mengambil beberapa bentuk yang berbeda agar tidak terjadi duplikasi *Object* Kita kenal sebagai *Polymorphism*.

Antara penurununan sifat (*inheritance*) maupun *Polymorphism* merupakan konsep yang memungkinkan digunakannya suatu *interface* yang sama untuk memerintah objek agar melakukan aksi atau tindakan yang mungkin secara prinsip sama namun secara proses berbeda. Dalam konsep yang lebih umum sering kali *Polymorphism* disebut dalam istilah tersebut.

6.2 Inheritance

Inheritance merupakan suatu cara untuk menurunkan suatu kelas yang lebih umum menjadi suatu kelas yang lebih spesifik. *Inheritance* adalah salah satu ciri utama suatu bahasa program yang berorientasi pada objek, dan Java pasti menggunakannya. Java mendukung *inheritance* dengan memungkinkan satu kelas untuk bersatu dengan kelas lainnya ke dalam deklarasinya. Dalam inheritance terdapat dua istilah yang sering digunakan. Kelas yang menurunkan disebut kelas dasar (*base class*/*super class*), sedangkan kelas yang diturunkan disebut kelas turunan (*derived class*/*subclass*). Karakteristik pada *superclass* akan dimiliki juga oleh *subclass*-nya.



Gambar 6- 1 Ilustrasi pada kelas manusia

Manusia adalah *superclass* dari pria dan wanita. Pria dan wanita mewarisi state dan *behaviour* kelas manusia.

Contoh pewarisan dalam program Java:

```
class A {
	int x;
	int y;
	void tampilXY() {
		System.out.println("nilai x: "+x+" nilai y: "+y);
	}
}
class B extends A {
	int z;
	void jumlahXY()
	{
		System.out.println("jumlah: "+(x+y+z));
	}
}
public class DemoInheritance {
	public static void main(String[] args) {
	A superclass=new A();
	B subclass=new B();
	System.out.println("superclass :");

	superclass.x=3;
	superclass.y=4;
	superclass.tampilXY();
	System.out.println("subclass :");

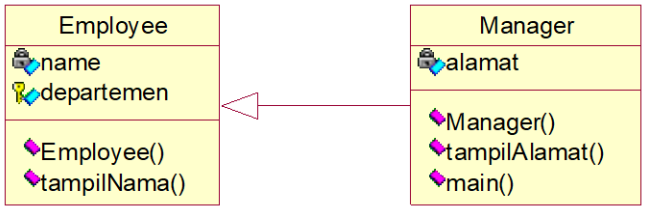
	//member superclass dapat diakses dari subclassnya
	subclass.x=1;
	subclass.y=2;
	subclass.tampilXY();
```

```
        //member tambahan hanya ada di subclass
        subclass.z=5;
        subclass.jumlahXY();
        }
}
```

6.3 Keyword Super

Keyword super digunakan untuk merefer *superclass* dari suatu kelas, yaitu untuk merefer *member* dari suatu *superclass*, baik atribut maupun *method*.

Contoh :



Gambar 6-2 Contoh kelas dengan keyword super

Dari *UML* diatas dapat dilihat bahwa kelas **Employee** merupakan *superclass* dari kelas **Manager**.

```
class Employee {
      private String name;
      String departemen;
      public Employee (String s) {
            name = s;
      }
      public void tampilNama()
      {
            System.out.println("nama : "+name);
      }
}
class Manager extends Employee {
      private String alamat;

      public Manager(String nama, String s) {
       /*memanggil konstruktor employee*/
            super(nama);
            alamat = s;
      }

      public void tampilAlamat() {
       /*menginisialisasi variabel departemen    yang ada pada
superclass*/
            super.departemen="Personalia";

       /*memanggil Method tampilNama() yang ada pada superclass*/
            super.tampilNama();
       /*menampilkan variabel departemen yang telah diinisialisasi*/
            System.out.println("alamat : "+alamat);
            System.out.println("departemen : "+super.departemen);
      }
```

```
      public static void main(String[] args) {
       /*membuat objek*/
            Manager adi = new Manager("adi","sukabirus");
                  adi.tampilAlamat();
      }
}
```

Output Program :

```
Nama : adi
Alamat : sukabirus
Departemen : Personalia
```

6.4 Overloading Method

Di dalam java Anda diperbolehkan membuat dua atau lebih *method* dengan nama yang sama dalam satu kelas, tetapi jumlah dan tipe data argumen masing – masing *method* haruslah berbeda satu dengan yang lainnya. Hal itu yang dinamakan dengan *Overloading Method*.

Contoh program:

```
class Lagu {
  String pencipta;
  String judul;

  void IsiParam(String param1) {
        judul    = param1;
        pencipta = "Tidak dikenal";
  }

  void IsiParam(String param1,String param2) {
       judul     = param1;
       pencipta  = param2;
  }



  void CetakKeLayar() {
     System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
  }
}
class DemoOver2 {
public static void main(String[] args) {
    Lagu d,e;
    d = new Lagu();
    e = new Lagu();

    d.IsiParam("Lagu 1");
    e.IsiParam("kepastian yang kutunggu","GiGi");


    d.CetakKeLayar();
    e.CetakKeLayar();
  }
}
```

Pada contoh program di atas, Kita melakukan *Overloading* terhadap *method* `IsiParam()`. Di mana `IsiParam()` yang pertama menerima satu buah parameter bertipe String dan `IsiParam()` yang kedua menerima dua buah parameter bertipe String. Sehinnga inilah hasil eksekusi program yang didapat:

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu: GiGi
```

6.5 Overiding Method

Di dalam java, jika dalam suatu *subclass* Anda mendefinisikan sebuah *method* yang sama dengan yang dimiliki oleh superclass, maka *method* yang Anda buat dalam *subclass* tersebut dikatakan meng-*override superclass*-nya. Sehingga jika Anda mencoba memanggil *method* tersebut dari *instance subclass* yang Anda buat, maka *method* milik *subclass*-lah yang akan dipanggil, bukan lagi *method* milik *superclass*-nya.

Contoh program:

```
class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}

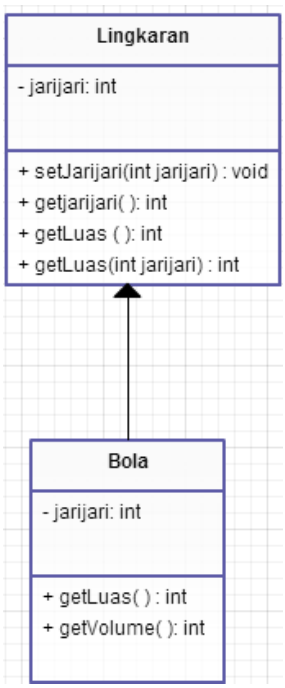
class B extends A {
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}

class DemoInheritance {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output program diatas:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

Terlihat pada contoh di atas, **kelas *B*** yang merupakan turunan dari **kelas *A***, meng-*override method* **tampilkanKeLayar()**, sehingga pada waktu Kita memanggil *method* tersebut dari variabel *instance* **kelas *B*** (variabel subOb), yang terpanggil adalah *method* yang sama yang ada pada **kelas *B***.





Perhatikan kelas diagram disamping. Jawablah pertanyaan dibawah ini!

a. Sebutkan *method* mana yang melakukan *override*?
b. Sebutkan *method* mana yang melakukan *overload*?
c. Buatlah *syntax* yang mengaplikasikan kelas diagram tersebut.

Fakultas Informatika
School of Computing
Telkom University
informatics lab